

The Art of Scientific Computation

A survey of techniques for simulation of the abelian sandpile model

James Reuben Bender

2024-10-26

Table of contents

test	3
1 Introduction	4
1.1 The abelian sandpile model	4
2 Implimenting a naive BTW simulation in different programming languages	6
2.1 Basic toppling and stabilisation algorithms	6
2.1.1 the ‘push’ topple algorithm	6
2.1.2 Naive stabilisation algorithm	7
2.2 Benchmarking setup	7
2.3 Results	7
3 Algorithms	9
3.0.1 Stabilisation algorithms	9
3.0.2 Targeted stabilisation algorithm	10
3.1 Performance comparison	11
4 References	12
Appendices	13
A julia	13
A.1 Appendix 2	13

test

1 Introduction

1.1 The abelian sandpile model

The abelian sandpile model is a cellular automaton inspired by the cascading behaviour of collections of sand grains piled to a critical height Bak, Tang, and Wiesenfeld (1988) . Often, and hereafter, referred to as the Bak-Tang-Weisenfeld (BTW) model after its early investigators Bak, Tang, and Wiesenfeld (1987), the model has become a prototypical example of ‘self-organised criticality’ in physical systems Dhar (1990) and has inspired numerous extensions (see J  rai (2018) for review).

The BTW model is defined on a lattice, where each point (‘site’) holds some number of sand grains (i.e. can be thought of as being at certain ‘height’). When a particular number of grains accumulate on a site, the pile becomes unstable and *topples* sending grains to its neighbors (itself losing that number of grains). This spread of sand grains can cause a chain reaction of topplings -an ‘avalanche’- that can propagate through the lattice. While each topple is simple and deterministic, predicting the size and shape of the avalanches on a large lattice with an arbitrary height distribution can be non-trivial - a problem compounded as we model the behaviour of the system over time (i.e. as we sequentially add grains). Therefore, the abelian sandpile model is a good candidate for computational simulation. By repeating the toppling/avalanche process iteratively, we may characterize features of the sandpile model (e.g. the distribution of avalanche sizes, the average number of grains present on the lattice, or any apparent equilibrium states) statistically rather than analytically.

This project presents several methods of simulating the BTW model.

When using a finite grid and losing grains at the edges, our output is subject to so called ‘finite size effects’, whereby extreme events are less common than we would expect on an infinite grid. This is because the avalanches are more likely to be cut off by the boundaries. To a ### Applications of the BTW model e.g. simulation of physical systems, traffic flow, forest fires, epidemiological models etc. ## Scientific programming tools

In large sandpiles, which have the potential for large scale avalanches (e.g. involving millions of topples), the computational tools and techniques used to simulate the model are crucial. A secondary aim of this project (in addition to presenting the BTW model) is to explore several alternative computational techniques for developing and simulating the model. There are two main aspects I will consider:

Algorithm design: The way in which the toppling and stabilisation algorithms implemented can have a significant impact on the efficiency of the simulation. These design decisions will depend on the backend on which the simulation is run - e.g. Does the algorithm complexity scale efficiently with the lattice size (to avoid memory bottlenecks)? Can the algorithm make use of the multithreading capabilities of modern CPUs (or, moreover, GPUs)? Therefore, there is unlikely to be a single optimal algorithm, but rather a trade-off between different factors. I will compare the performance of several different algorithms for simulating the BTW model.

Programming languages: The choice of programming language can also have a significant impact on the efficiency of simulation; both in the sense of the time taken to run the simulation, and the ease of developing and maintaining code. Modern scientific programming typically falls into a two-language paradigm, wherein a relatively high-level, dynamic language (usually python) is used for rapid prototyping and data analysis, and a lower-level, statically typed and compiled language (e.g. C, C++, Fortran) is used for computationally intensive tasks. There is some middle ground here with the use of high level wrappers in dynamic languages that call high performance compiled code for performance critical tasks (e.g. numpy, scipy, numba in python).

While these approaches well established and can result in highly efficient implementations, the two language approach has drawbacks. The most obvious is the need to develop (and potentially maintain) two separate codebases. Moreover, the specialised skills needed to develop and maintain high performance code in a low-level language can be a barrier to entry for many researchers. This barrier can be compounded by the discontinuity in source material between two codebases. For example, localising and addressing bugs can require more elaborate error handling and logging when the code is split between two languages.

The Julia language was developed as an alternative to the two-language approach @. A high level, dynamic language Julia's progressive typing and type inference system, along with just-in-time compilation, intends to allow for easily readable and maintainable code that can be optimised for performance without leaving the language - a language that 'walks like python, runs like C'.

This report will present a comparison of the performance of several different algorithms for simulating the BTW model, implemented in Julia and python. The aim is to explore the trade-offs between algorithm design and programming language choice, and to potentially to inform students or researchers developing their own simulations of the BTW model.

2 Implimenting a naive BTW simulation in different programming languages

2.1 Basic toppling and stabilisation algorithms

2.1.1 the ‘push’ topple algorithm

The push algorithm is a simple way to implement the toppling of a site in a sandpile model. The implimentation follow the intuitive desription of the toppling process: if a certain site X_{ij} is at/above the height threshold k , and so is unstable. stabilisation of this site occurs by distributing its grains to it's four neighbours. i.e. each of $X_{i-1,j}$, $X_{i+1,j}$, $X_{i,j-1}$, $X_{i,j+1}$ recieves one grain and X_{ij} looses four grains. Figure 2.1

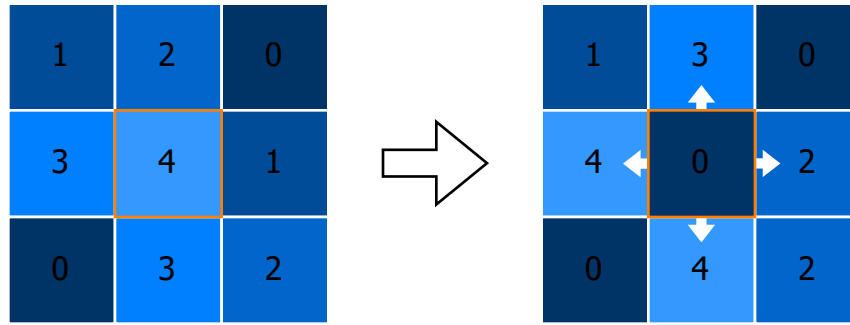


Figure 2.1

Algorithm 2.1 Push toppling algorithm

```

1: if  $X_i \geq k$  then
2:    $Y_i \leftarrow X_i - k$ 
3:   for each neighbor  $j$  of  $i$  do
4:      $Y_j \leftarrow X_j + 1$ 
5:   end for
6: else
7:    $Y_i \leftarrow X_i$ 
8: end if

```

2.1.2 Naive stabilisation algorithm

The naive stabilisation algorithm is naive in the sense that it does not assume any information about where topples might occur (i.e. it checks every site, c.f. the targeted algorithm). The algorithm iterates over each site in the lattice applying a toppling rule (e.g. the push algorithm) until no sites are above the threshold.

Algorithm 2.2 Naive stabilisation algorithm

```
1: while any site in  $X$  is above  $k$  do  
2:   for each site  $i$  in  $X$  do  
3:     if  $X_i \geq k$  then  
4:        $Y \leftarrow \text{topple!}(X, i)$   
5:     else  
6:        $Y_i \leftarrow X_i$   
7:     end if  
8:   end for  
9: end while
```

2.2 Benchmarking setup

To compare the performance across languages, a consistent test setup is needed. Thankfully, the deterministic nature of the sandpile model allows the simple construction of sandpiles with known stabilisation times. Given a $N \times N$ lattice, test sandpiles can be constructed by filling the central $3 < n < N$ sites of an empty lattice with $k - 1$ grains. addition of a single grain at the center of the lattice will cause increasing number of topplings as n increases. Such regular grids will be used to test the performance of implementations of the naive push stabilisation algorithm in python, and julia. Specific implimentations of the algorithm in each language are given in appendix 1.

2.3 Results

The performance of the naive stabilisation algorithm was tested on a 101×101 lattice with $k = 4$ grains. The number of topplings required to stabilise the lattice was recorded for each n in the range $3 < n < 100$. The results are shown in Figure [2.2](#)

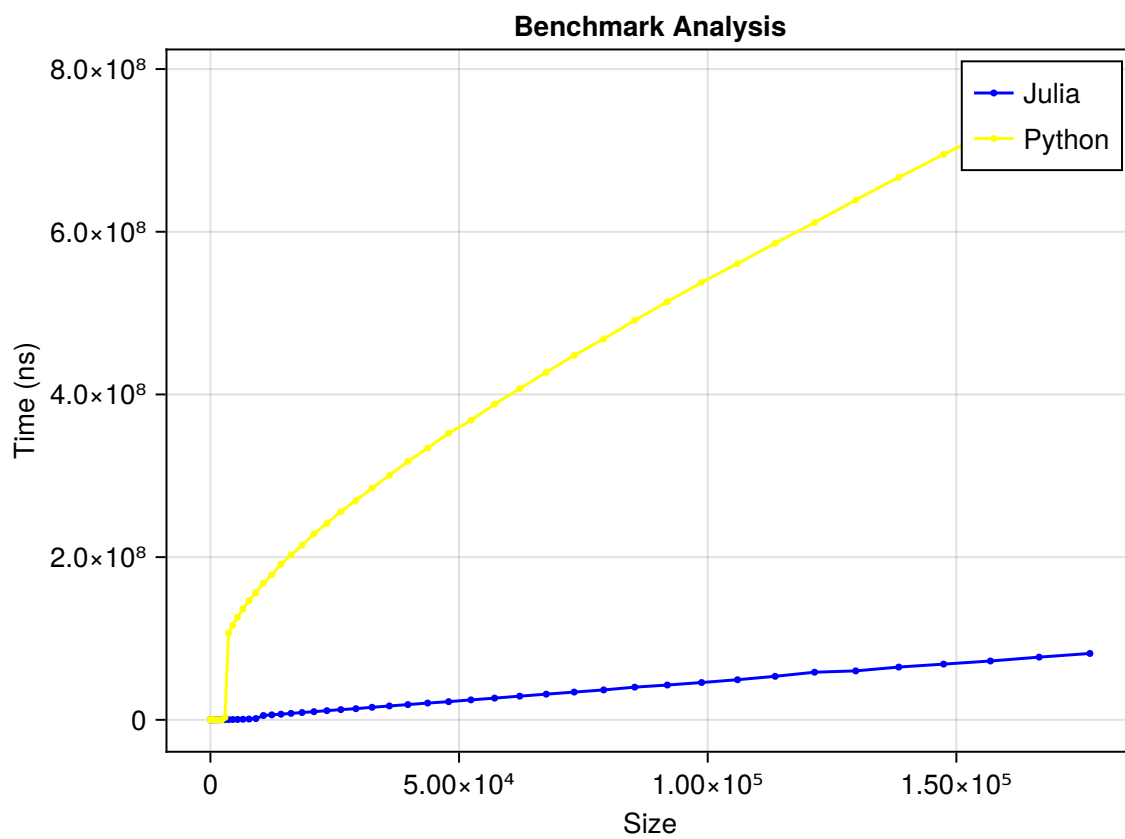


Figure 2.2

3 Algorithms

Because the every topple event using the naive algorithm effects the output of five locations on the lattice, the algorithm is not suitable for parallelization: the order of the for loop must be preserved to prevent multiple threads from updating the same site simultaneously (which would potentially lead to conflicting results when checking if a site is above the threshold). To remedy this limitation, we can use the parallel, or ‘pull’ algorithm

3.0.0.1 ‘pull’ topple algorithm

The parallel is based on the idea of “pulling” sand from neighbors rather than “pushing” sand to neighbors. Each cell is updated by checking each of it’s neighbours - for each neighbour that is ready to topple, the cell gains one grain. If the cell itself is ready to topple, it loses four grains. Each iteration of the for loop therefore only changes one site (i.e. each iteration is encapsulated), and therefore each cell can be updated in parallel.

Algorithm 3.1 Pull toppling algorithm

```
1: if  $X_i \geq k$  then  
2:    $Y_i \leftarrow X_i - k$   
3: else  
4:    $Y_i \leftarrow X_i$   
5: end if  
6: for each neighbor  $j$  of  $i$  do  
7:   if  $X_j \geq k$  then  
8:      $Y_i \leftarrow Y_i + 1$   
9:   end if  
10: end for
```

3.0.1 Stabilisation algorithms

3.0.1.1 Naive stabilisation algorithm

The naive stabilisation algorithm is naive in the sense that it is the most straightforward way to implement the toppling algorithm, but also in the sense that it does not assume any information about where topples might occur. It iterates over each site in the lattice, and if

the site has more sand than the threshold, it topples. This loop is repeated until each cell is below the toppling threshold. This algorithm works effectively with both push and pull toppling algorithms.

Algorithm 3.2 Naive stabilisation algorithm

```

1: while any site in  $X$  is above  $k$  do
2:   for each site  $i$  in  $X$  do
3:     if  $X_i \geq k$  then
4:        $Y \leftarrow \text{topple!}(X, i)$ 
5:     else
6:        $Y_i \leftarrow X_i$ 
7:     end if
8:   end for
9: end while

```

3.0.2 Targeted stabilisation algorithm

This algorithm assumes we have information about the locations of potential topple sites. For example, when dropping a single grain of sand, we first can check if the drop site needs to topple. If it doesn't, there is no possibility of avalanche and no other sites need to be checked. If it does, can topple that site and just check the the neighbors to which the sand spreads. While needing less checks, we now must keep track of the sites that need to be checked. It is expected that this algorithm will provide performance improvements when the number of sites that need to be checked is significantly less than the total number of sites (i.e. a 'sparse' avalanche)

Algorithm 3.3 Targeted toppling algorithm

```

1: while  $L$  is not empty do
2:   remove element  $i$  from  $L$ 
3:   if  $X_i \leq k$  then
4:      $Y \leftarrow \text{topple!}(X, i)$ 
5:   end if
6: end while

```

3.1 Performance comparison

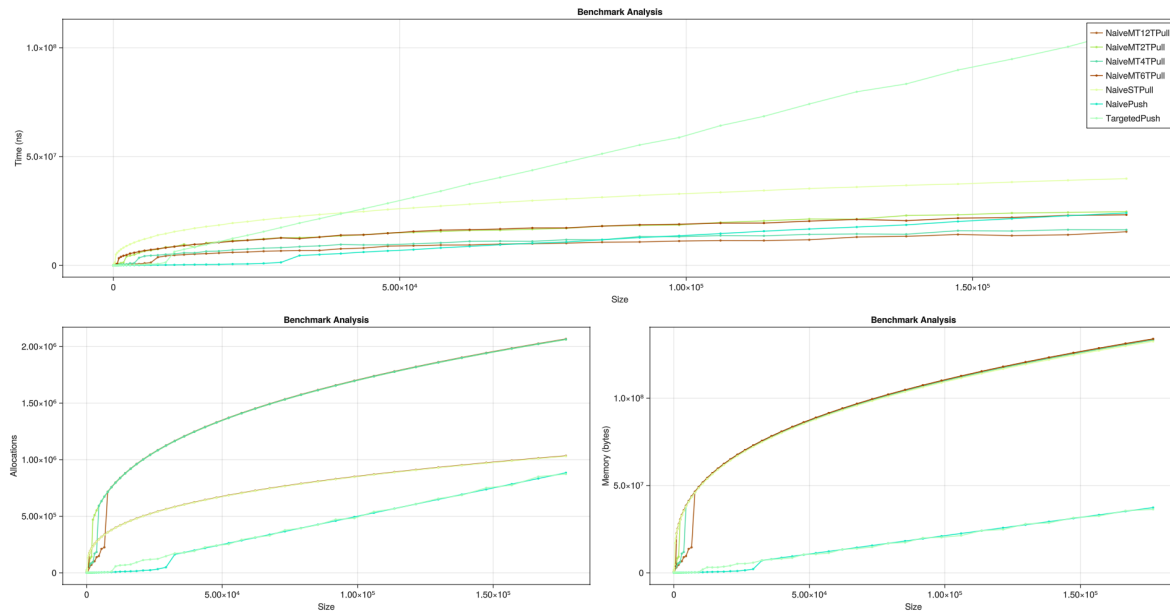


Figure 3.1: Performance comparison

4 References

- Bak, Per, Chao Tang, and Kurt Wiesenfeld. 1987. “Self-Organized Criticality: An Explanation of the $1/f$ Noise.” *Physical Review Letters* 59 (4): 381–84. <https://doi.org/10.1103/PhysRevLett.59.381>.
- . 1988. “Self-Organized Criticality.” *Physical Review A* 38 (1): 364–74. <https://doi.org/10.1103/PhysRevA.38.364>.
- Dhar, Deepak. 1990. “Self-Organized Critical State of Sandpile Automaton Models.” *Physical Review Letters* 64 (14): 1613–16. <https://doi.org/10.1103/PhysRevLett.64.1613>.
- Járai, Antal A. 2018. “Sandpile Models.” *Probability Surveys* 15 (January). <https://doi.org/10.1214/14-PS228>.

A julia

A.1 Appendix 2

A.1.0.1 Test